

Operation TELESCREEN - Instructor Solution Key

LEGAL DISCLAIMER: The information, tools, and code provided in this repository and course are strictly for educational, research, and defensive purposes only.

You are explicitly prohibited from using any materials contained herein to access, test, modify, or exploit any device, network, or system that you do not own 100% or for which you do not have explicit, documented, and legally binding authorization to interact with.

By using this repository and course, you acknowledge and agree that:

1. Any illegal, unauthorized, or malicious use of this information is solely your responsibility.
2. The author(s) and contributor(s) of this repository and course shall not be held liable for any damages, legal repercussions, criminal charges, or unauthorized actions resulting from the use, misuse, or abuse of the contents herein.
3. You will comply with all applicable local, state, national, and international laws regarding cybersecurity and computer fraud.

IF YOU DO NOT AGREE WITH THESE TERMS, DO NOT USE THIS REPOSITORY AND COURSE.

Artifact Identity

The instructor-issued artifact hashes are:

TELESCREEN-full.img
037cc979a58f04f3285132a7b745ca859b1604aeb80c9ef62eb5377e48bd18ec
TELESCREEN-boot.img
4193d6a9e9a29848813056f05356c17e4a4e64f8206148c86fd3b0d50a5e69fd
TELESCREEN-env.img
e935f998fd6289615d88d8d2c2d4d3aada0e04ad6dd7592c27a853431fec4d7e
TELESCREEN-kernel.img
8a5fb5725420d79ae4e21e39e27c161b7df88dbbdbc2b061d859dd889c8b0230
TELESCREEN-rootfs.img
38a8684afb5f8bb8b717b2ab333dc729054d36192aa6541be6e9fed6f19cd10e

Task 1: Carve and Identify the Four Partitions (10 points)

Solution

Expected Partition Layout

The environment declares the layout; decode it from mtdparts:

```
mtdparts=sfc:128K(boot),64K(bootargs),1792K(kernel),14400K(rootfs)
```

mtd	name	offset	size	end
0	boot	0x000000	0x020000 (128 KiB)	0x020000
1	bootargs	0x020000	0x010000 (64 KiB)	0x030000
2	kernel	0x030000	0x1C0000 (1792 KiB)	0x1F0000
3	rootfs	0x1F0000	0xE10000 (14400 KiB)	0x1000000

Coverage: $0x1F0000 + 0xE10000 = 0x1000000$ - the whole 16 MiB chip.

Carving

```
dd if=TELESCREEN-full.img of=boot.img bs=1 count=$((0x20000))
dd if=TELESCREEN-full.img of=env.img bs=1 skip=$((0x20000))
count=$((0x10000))
dd if=TELESCREEN-full.img of=kernel.img bs=1 skip=$((0x30000))
count=$((0x1C0000))
dd if=TELESCREEN-full.img of=rootfs.img bs=1 skip=$((0x1F0000))
count=$((0xE10000))
```

Magic-Byte Identification

Partition	First bytes	Meaning
boot	15 05 00 ea	ARM B (reset vector) at offset 0
bootargs	CRC32 (4 B, LE) then baudrate=...	U-Boot environment
kernel	21 84 1b 00 ... 67 7a 69 70 68 65 61 64	vendor container + "gziphead"
rootfs	85 19 03 20	JFFS2 little-endian magic + CLEANMARKER

Grading Rubric (1-to-1 Mapping)

Criterion	Points	Full Credit (Answer Key)
	2	Matches the instructor-issued hash

Criterion	Points	Full Credit (Answer Key)
[DOCUMENT] Whole-image hash verified		
[DOCUMENT] Partition table correct	3	Offsets 0x0/0x20000/0x30000/0x1F0000, sizes 128K/64K/1792K/14400K, mtdparts decoded
[DOCUMENT] Magic-byte identification	3	All four magics correct
[DOCUMENT] Carved files produced	2	Four files, correct sizes

Instructor Notes & Assembly

- The **1984 KiB pad** between the end of kernel and the start of rootfs (0x1F0000 – 0x30000 – 0x1C0000 = 0x1F0000) is deliberate vendor padding; students who “fix” the gap by shifting rootfs are wrong.
- If a student identifies boot by the U-Boot string instead of the vectors, that is acceptable, but the **vectors at offset 0** are the primary answer.

Task 2: Reverse the Boot Chain (15 points)

Solution

First-Stage Entry

The vector table at offset 0 of boot:

```
00000000: 15 05 00 ea b 0x145c ; reset
00000004: fe ff ff ea b . ; undef
00000008: fe ff ff ea b . ; svc
...
```

Reset handler entry: **0x145c** (relative to the boot partition).

Cold-Boot Path (representative)

```
0000145c: ... ; SVC mode, IRQ/FIQ off, VBAR=0
000014bc: ... ; warm/cold magic check at 0x12020140
000014d8: bl 0x17e4 ; DRAM / parameter init
000014e0: bl 0x1ae8 ; SoC / clock init
000014f4: mov pc, r1 ; jump to U-Boot payload
```

U-Boot

U-Boot 2024.07 (telescreen-rp5)

Environment

Decode bootargs and bootcmd from env.img:

```
bootargs=mem=47M console=ttyAMA0,115200 root=/dev/mtdblock3
rootfstype=jffs2 rw \
```

```
mtdparts=sfc:128K(boot),64K(bootargs),1792K(kernel),14400K(rootfs)
lpj=9838592
bootcmd=sf probe 0;telescreenapp;boothz 0x41000000 0x40008000
0x30000 0x1C0000
```

bootcmd decodes to: probe SPI, run the vendor pre-boot hook, then load the kernel from flash offset **0x30000** with size **0x1C0000** (the kernel partition).

Grading Rubric (1-to-1 Mapping)

Criterion	Points	Full Credit (Answer Key)
[DOCUMENT] First-stage entry point	4	0x145c (reset vector target)
[DOCUMENT] Cold-boot path traced	4	At least two of: 0x17e4 (DRAM), 0x1ae8 (SoC)
[DOCUMENT] U-Boot version/build	3	U-Boot 2024.07 (telescreen-rp5)
[DOCUMENT] bootargs/bootcmd decoded	4	Both strings, including mtdparts and the boothz load

Instructor Notes & Assembly

- The **BootROM is silicon** and is *not* in the image; students should not expect to find it. The first bytes in the image are the **SPL** (first stage), which the BootROM loads.
- The warm/cold magic at 0x12020140 selects DRAM init; if a student notes it, award full marks on the cold-boot criterion.

Task 3: Inflate the Kernel and Read the Device Tree (15 points)

Solution

Container Header (kernel.img)

```
00000000: 21 84 1b 00    magic    0x001B8421
00000004: 00 f0 2c 00    length  0x002CF000
00000008: 67 7a 69 70 68 65 61 64    "gziphead"
```

```
00000010: 1f 8b 08 08    gzip magic, FNAME flag
0000001a: 49 6d 61 67 65 00    "Image\0"
```

Inflate

```
dd if=kernel.img bs=1 skip=16 | gunzip > Image
# raw Image size: 2945024 bytes (0x2CF000)
```

Device Tree and Banner

```
model = "telescreen,rp5 DEM0 Board";
compatible = "telescreen,rp5";

Flash-controller node (from the DTB): telescreen,fmc with child
telescreen,fmc-spi-nor and a sfc node compatible = "jedec,spi-nor".

Kernel banner:

Linux version 4.9.37 (super@super-virtual-machine) (gcc version
7.3.0 (GCC))
#1 Sat Nov 9 13:45:32 CST 2024
```

Grading Rubric (1-to-1 Mapping)

Criterion	Points	Full Credit (Answer Key)
[DOCUMENT] Container header parsed	5	Magic 0x001B8421, length, gziphead
[DOCUMENT] Raw Image produced	5	Size 2945024, first bytes 04 90 8f e2
[DOCUMENT] Device tree + banner	5	Model telescreen,rp5 DEM0 Board, flash node, Linux 6.6

Instructor Notes & Assembly

- The container is **not** a uImage (0x27051956) or a zImage (0x016F2818); the vendor tag gziphead is the giveaway.
- The DTB is appended in the kernel partition near offset 0x1b8434 (size 12676, FDT v17). Students may extract it from the partition rather than the inflated image.

Task 4: Open the Rootfs (15 points)

Solution

Extraction

```
jefferson -d rootfs rootfs.img
```

The tree contains the application, the web directory, and the config directory.

The Two CRC Algorithms

Layer	Algorithm	Coverage
U-Boot env	crc32 (standard, init 0xFFFFFFFF, final xor)	the key=value\0 blob, stored at offset 0
JFFS2 node	crc32_le (reflected, no init/final inversion)	hdr_crc=node[0:8], node_crc=node[0:60], data_crc=data

In-Place Patch (worked example)

Append a higher-version INODE for the target file (version = old + 1000), fill the remainder with a PADDING node so the total length is unchanged, then recompute in order: hdr_crc -> data_crc -> node_crc.

before: hdr_crc=0x... data_crc=0x... node_crc=0x...

after: hdr_crc=0x... data_crc=0x... node_crc=0x... (all re-verified)

Grading Rubric (1-to-1 Mapping)

Criterion	Points	Full Credit (Answer Key)
[DOCUMENT] Rootfs extracted	4	Application + web dir + config dir listed
[DOCUMENT] CRC algorithms compared	6	Both named; JFFS2 uses crc32_le (no init/final), env uses standard CRC32
[DOCUMENT & PATCH] In-place patch verified	5	Patch with before/after CRC values for all three JFFS2 CRCs

Instructor Notes & Assembly

- The most common error is using zlib's CRC for JFFS2. If the patched node fails to mount, this is why.
- Accept a patch that stores the replacement **uncompressed** (compr=0) as long as the total node length is preserved and the CRCs verify.

Task 5: Find the Backdoors (20 points)

Solution

B1 - Config-Sourced Root Execution

The init script sources a writable configuration file **as root**:

```
# /etc/starts -> /mnt/mtd/ipc/run
. $WIFIPATH           # sources /mnt/mtd/ipc/conf/wifi.conf as root
```

Exploit: any write to that config file executes as root on the next boot.

B2 - CGI Dispatcher Command Injection

The HTTP dispatcher (HI_CGI_Interface in the application) routes internal *.cgi endpoints. One handler builds a shell command from a request parameter and calls system(). Exploit: GET /<route>?... with shell metacharacters -> root command execution.

B3 - Archive-to-Root Restore

The restore handler stages an uploaded archive and runs:

```
tar -xvzf /mnt/mtd/ipc/tmpfs/config_restore.bin -C /
```

Exploit: an archive containing ../ entries writes anywhere on the filesystem **as root**. The path is fixed; the injection is the **archive contents**.

B4 - Credential Store

The web credential file is empty and the device ships default credentials; the login succeeds with defaults or no password.

B5 - Debug Root Shell

A debug path (a script or a service) provides a root shell locally (e.g. a serial getty or a triggered shell). Exploit: local/console root.

Grading Rubric (1-to-1 Mapping)

Criterion	Points	Full Credit (Answer Key)
[DOCUMENT] B1	4	. \$WIFIPATH in the init path + exploit line
[DOCUMENT] B2	4	Dispatcher + at least one system() site
[DOCUMENT] B3	4	tar ... -C / + network reachability proof
[DOCUMENT] B4	4	

Criterion	Points	Full Credit (Answer Key)
		Empty/default credential store demonstrated
[DOCUMENT] B5	4	Debug shell + trigger

Instructor Notes & Assembly

- B2 and B3 are the highest-value remote-root bugs; B1 is the design flaw that makes “config write” equal “root exec”.
- If a student proves B3 end-to-end with a crafted archive, award the full B3 points even if they do not name the exact function.

Task 6: Break the Exfiltration Crypto (15 points)

Solution

The Key Schedule

The beacon key is derived by **MD5 over a concatenation of device identifiers**:

```
sprintf(buf, "%s&%s*%s", id1, id2, id3);
HI_P2P_MD5_Get(buf, md5hex);          /* 32 hex chars */
/* a selection loop reduces md5hex into the key bytes */
```

i.e. `key = f(MD5("id1 & id2 * id3"))`. Because the identifiers are **public** (printed on the device / readable in the config), the key is **derivable by anyone**.

Decryptor (worked example)

```
import hashlib
def key_for(id1, id2, id3):
    return hashlib.md5(f"{id1}&{id2}*{id3}".encode()).hexdigest()
```

Running the decryptor on a captured beacon yields the readable telemetry.

200-Word Analysis (answer key)

A key derived from a public identifier is **obfuscation, not encryption**. The confidentiality of a cipher rests entirely on the secrecy of its key; if the key is a deterministic function of data an attacker already has (the device’s public ID, a serial, a broadcast DID), then the attacker can recompute the key and decrypt every message. The cipher may be strong - even AES - and the scheme still provides **zero confidentiality**. This is the failure in the TELESCREEN exfiltration channel: the “sealed” payloads are sealed with a key that the device hands out in the clear. Correcting it requires a **secret** key: either a pre-shared secret provisioned out of

band, or a per-session key agreed with a real key-exchange (X25519) and derived with a KDF (HKDF). Integrity must also be provided by an **AEAD** so forged beacons are rejected. Anything less is theatre.

Grading Rubric (1-to-1 Mapping)

Criterion	Points	Full Credit (Answer Key)
[DOCUMENT] Key schedule recovered	6	key = f(MD5(id1 & id2 * id3))
[PATCH] Decryptor works	5	Decrypts a captured beacon to readable telemetry
[DOCUMENT] 200-word analysis	4	Explains that a public-ID key gives no confidentiality

Instructor Notes & Assembly

- The MD5 here is a **reduction step**, not the vulnerability by itself; the vulnerability is the **public key source**.
- Award full marks for a correct formula in any equivalent notation.

Task 7: Build the RP5 TELESCREEN-Lab (10 points)

Solution

Four-Partition Layout

Lay out the RP5 image store with the **same offsets and sizes**:

boot	0x000000	128 KiB
bootargs	0x020000	64 KiB
kernel	0x030000	1792 KiB
rootfs	0x1F0000	14400 KiB

Boot

Build/boot U-Boot on RP5; have it load the kernel container from the store, inflate it, and jump to the Linux Image. The kernel command line carries the same mtdparts.

Router Proof

wlan0 in AP mode (hostapd) + eth0 WAN + NAT. A client associates and receives DHCP through the AP, then reaches the WAN.

AEAD Proof

Replace the weak beacon with **AES-256-GCM** (ARMv8 crypto extensions). A forged frame is rejected at the authentication tag.

Grading Rubric (1-to-1 Mapping)

Criterion	Points	Full Credit (Answer Key)
[PATCH] Four-partition layout reproduced	3	Same offsets/sizes as the TELESCEEN
[PATCH] RP5 boots the image	3	Serial shows U-Boot -> Linux -> app
[DOCUMENT] Router proof	2	Client gets DHCP via the AP and reaches WAN
[DOCUMENT] AEAD proof	2	Beacon is AES-256-GCM; forged frame rejected

Instructor Notes & Assembly

- The **format parity** is the point: if the RP5 image cannot be carved by the same dd commands as the camera, the task is not complete.
- Accept either AES-256-GCM or XChaCha20-Poly1305 for the AEAD proof, provided nonce discipline is demonstrated.

Task 8: The Defensive Playbook (bonus, 10 points)

Solution

- **Beacon detection:** periodicity/jitter analysis on outbound flows; payload-entropy analysis; flag the fixed collector endpoint.
- **Backdoor detection:** monitor the sourced config file for writes; block/lock the restore endpoint; alert on `system()`-invoking config changes; audit the credential store.
- **Field hardening:** secure boot + signed images; remove default credentials; replace the public-ID KDF with a secret-key AEAD; read-only rootfs with signed updates.

Grading Rubric (1-to-1 Mapping)

Criterion	Points	Full Credit (Answer Key)
[DOCUMENT] Beacon detection	4	Periodicity + entropy, with a concrete method
[DOCUMENT] Backdoor detection	3	At least two concrete indicators
	3	

Criterion	Points	Full Credit (Answer Key)
[DOCUMENT] Field hardening		Secure boot, signed images, key management

Written Analyses (Q1-Q3)

Q1 - The Tradecraft Parallel (5 points)

Full credit: specific parallels to real backdoored cameras/routers; explains why a “sealed” channel built on a public-ID key fails.

Q2 - Ethics and Law (5 points)

Full credit: balanced; discusses authorisation, proportionality, and disclosure.

Q3 - Defensive Recommendations (5 points)

Full credit: one hardware, one firmware, one operational measure, each specific.

Task 6 - Reverse the Stripped Node Binary (25 points)

Solution

The node application is `firmware/ctfnode.stripped` - a stripped Linux aarch64 ELF. Every defect is a named function in the source and a `FUN_00xxxxxx` in the binary. This is the mapping the student must recover with Ghidra.

defect	function	address	stripped label	how it is found
B6	ctf_weak_key	0x004009b0	FUN_004009b0	calls <code>ctf_crc32_le</code> in a 32-iteration loop over the UID
B4	ctf_login	0x00400a8c	FUN_00400a8c	references the strings <code>admin</code> and <code>""</code> ; two <code>strcmp</code> calls
B1	ctf_config_run	0x00400ae0	FUN_00400ae0	contains the literal <code>run=</code> and calls <code>system</code>
B2	ctf_build_cmd	0x00400b64	FUN_00400b64	<code>snprintf(..., "ping -c 1 %s", value)</code>

defect	function	address	stripped label	how it is found
B2	ctf_http_handle	0x00400b80	FUN_00400b80	calls ctf_build_cmd then system
B3	ctf_restore	0x00400bc0	FUN_00400bc0	snprintf(..., "tar -xvzf %s -C /", ...) then system
B5	ctf_debug_shell	0x00400c00	FUN_00400c00	system("/bin/ sh")
-	ctf_crc32_le	0x00400960	FUN_00400960	the 0xEDB88320 polynomial constant
-	ctf_banner	0x00400c0c	FUN_00400c0c	the string TELESCREEN node – the wall unit sees you
-	ctf_dispatch	0x00400c20	FUN_00400c20	strcmp chain on argv[1]; calls every feature

Method (the four resolution rules)

- **R1** exact address match against firmware/ctfnode.unstripped (the key),
- **R2** .plt stub -> JUMP_SLOT -> import name (system, snprintf, strcmp, fopen, fgets, ...),
- **R3** the .plt PLT0 resolver (here 0x004006f0),
- **R4** phantom functions on alignment padding.

Note ctf_crc32_byte does **not** appear as a function: the compiler inlined it into ctf_crc32_le. Recognising inlining is part of the task.

Key check: the weak KDF uses the **JFFS2-style crc32_le** (reflected, no final XOR). The expected key for the demo UID SSAT-468547-FEEBD is da506e04af00c6f40394d2cd2295bfc8682e8b9f9e9b844cea50c08d5f483141, and scripts/weak_decrypt.py --uid SSAT-468547-FEEBD reproduces it.

Grading Rubric (1-to-1 Mapping)

points	for
10	every application function named and tied to a rule (R1-R4)
10	each defect B1-B6 located at its address with the offending call shown

points	for
5	the system / snprintf / strcmp import cross- references documented

Instructor Notes & Assembly

- Reference output: ghidra/RESOLUTION_MAP.md, CTF-XX-J-ghidra-function-resolution.md, and ghidra/proj/CTFNodeRE.gpr.
- Rebuild the target with ./firmware/build_target.sh; open with ./ghidra/make_project.sh.
- The CTF node is intentionally free of stack canaries only in spirit; the defects are logic flaws, not memory corruption, so a canary would not help.

Reference Material

- ARM Cortex-A76 TRM; ARMv8-A Cryptography Extensions
- Ghidra: <https://ghidra-sre.org/>
- JFFS2 (mtd-utils, jffs2dump)
- U-Boot environment / mtdparts
- NIST SP 800-38D (GCM); RFC 8439 (ChaCha20-Poly1305); RFC 7748 (X25519)